

# Assignment 1

## Building a Modern ELT Pipeline

Amna Raza Khan and Shumaila Javed, SBASSE, LUMS

### Assignment Overview

This practical assignment focuses on implementing a complete ELT pipeline that integrates multiple real-world data sources. You will extract data, store it in modern formats, perform transformations and quality checks, conduct exploratory analysis, and reflect on how your pipeline could support AI system development.

### Learning Objectives

Upon successful completion, you will be able to:

- Design and implement a modular ELT pipeline
- Extract data from APIs, public datasets, and web sources
- Handle structured, semi-structured, and unstructured data
- Store data in industry-standard formats (CSV, JSON, Parquet)
- Perform exploratory data analysis with appropriate visualizations
- Articulate how engineered datasets can support AI/ML applications

### Thematic Domain Selection

Choose **one thematic domain** to guide your data collection and analysis. This theme should provide coherence across all data sources and analyses.

- **Climate Technology:** Renewable energy adoption, carbon markets, sustainable tech
- **AI Labor Markets:** Job trends, skill demands, remote work evolution

- **Digital Health:** Telemedicine adoption, mental health tech, wearable devices
- **Financial Technology:** Digital payments, blockchain applications, regtech
- **Smart Mobility:** EV infrastructure, micromobility, autonomous vehicles
- **Cybersecurity:** Threat landscapes, privacy technologies, incident trends

**Note:** Your chosen theme will influence data source selection and should be maintained consistently throughout the assignment.

## Part 1: Extract and Load (EL) Implementation

### Required Data Sources

Your pipeline must integrate at least **three** of the following sources:

#### 1. API Data Source (Required)

- *Options:* Reddit (PRAW)

##### Reddit API (PRAW) Instructions:

- Create or log in to a Reddit account.
- Visit <https://www.reddit.com/prefs/apps>.
- Click *Create another app*.
- Select **script** as the application type.
- Use `http://localhost` for both the About URL and Redirect URI.
- Copy the generated **Client ID** and **Client Secret**.
- Store credentials using environment variables (do not hard-code keys).
- Access Reddit data in Python using the **praw** library.

- Extract relevant posts, articles, or time-series data related to your theme

#### 2. Public Dataset (Required)

- *Options:* Kaggle, government portals (data.gov), EU Open Data, etc.

**Kaggle Dataset Instructions:**

- Create or log in to a Kaggle account.
- Generate a Kaggle API token from the account settings page.
- Download datasets manually or using the Kaggle API.
- Store downloaded datasets locally within the project directory.
- Load datasets into the pipeline using Pandas.

- Should provide structured or semi-structured data relevant to your theme

### 3. Time-Series/Search Data (Required)

- *Options:* Google Trends (pytrends), financial data (yfinance), etc.

**Google Trends (pytrends) Instructions:**

- Install and use the `pytrends` library.
- No API authentication is required.
- Define keywords relevant to the thematic domain.
- Extract time-series search interest data.
- Requests may be subject to rate limiting.

**Yahoo Finance (yfinance) Instructions:**

- Install and use the `yfinance` library.
- Select relevant companies or financial instruments.
- Extract historical price or volume time-series data.
- No API authentication is required.

- Extract temporal patterns or popularity metrics

## Implementation Requirements

### (a) Modular Code Structure:

- Organize your code into separate modules/functions (e.g., `extract_api.py`, `load_data.py`)

- Use a main script (`run_pipeline.py`) or notebook to orchestrate the workflow
- Implement configuration management (API keys, parameters) using environment variables or a config file

(b) **Data Storage:**

- Store **raw extracted data** in a `data/raw/` directory
- Convert and store processed data in **formats:**
  - **CSV** - For human readability and basic analysis
  - **JSON** - For semi-structured data preservation
- Store processed data in a `data/processed/` directory

(c) **Basic Documentation:**

- Include a `README.md` explaining how to run your pipeline
- Document data source parameters and any assumptions made

## Part 1 Deliverables

- Working EL pipeline code with modular structure
- Raw data files for each source
- Processed data in CSV, JSON formats
- Pipeline architecture diagram (can be hand-drawn or digital)
- Brief documentation of your pipeline structure

## Part 1 Questions

- (a) **Data Heterogeneity:** Explain how your chosen data sources represent different data types (structured, semi-structured, unstructured). Provide concrete examples from your extracted data.
- (b) **Extraction Challenges:** Discuss specific technical or practical challenges encountered while accessing different data sources (rate limits, authentication, data format inconsistencies, etc.).
- (c) **Storage Justification:** Explain why storing data in multiple formats (CSV, JSON) is valuable in a data engineering context. When would you choose one format over another?

## Part 2: Transform, Clean, and Analyze

In this phase, you will transform the loaded data, assess its quality, perform exploratory analysis, and prepare it for potential use.

### Data Quality and Transformation

Using **Pandas**, perform the following tasks on at least two of your datasets:

(a) **Data Quality Assessment:**

- Calculate and report key quality metrics for each dataset:
  - Number of missing values
  - Number of duplicate records

**Hint: Data Quality Checks**

Pandas provides built-in functions to assess data quality:

- Check missing values using `isnull()` or `isna()`
- Count missing values using `sum()`
- Detect duplicate rows using `duplicated()`

(b) **Transformation and Cleaning:**

- Handle missing values appropriately (justify your approach)
- Remove or handle duplicate records
- Standardize data formats (dates, categorical values, etc.)
- Generate summary statistics for numerical columns

**Hint: Data Cleaning Operations**

Common Pandas functions for cleaning data include:

- `dropna()` or `fillna()` for handling missing values
- `drop_duplicates()` for removing duplicate records
- `to_datetime()` for standardizing date formats
- `describe()` for summary statistics

## Exploratory Analysis and Visualization

### Hint: Visualization Libraries

The following libraries may be used for visualization:

- `matplotlib` for basic plots
- `seaborn` for statistical visualizations

#### (a) Temporal Analysis:

- Create at least one time-series visualization showing trends over time

#### (b) Categorical/Frequency Analysis:

- Create at least one bar chart or histogram for categorical/frequency data

#### (c) Correlation/Relationship Analysis:

- Create at least one visualization exploring relationships between variables

## Part 2 Deliverables

- Cleaned, transformed datasets (store in `data/cleaned/`)
- At least three meaningful visualizations
- Jupyter notebook or Python script documenting your analysis

## Part 2 Questions

- Cleaning Rationale:** Justify your data cleaning decisions. Why were specific approaches chosen for handling missing data or outliers?
- Visualization Insights:** What key insights or patterns emerge from your visualizations? How do they relate to your chosen thematic domain?
- Visualization Critique:** What limitations exist in your current visualizations? How could they be improved for different audiences (technical vs. business stakeholders)?

# Grading Rubric

| Criterion                                   | Weight      | Max Points |
|---------------------------------------------|-------------|------------|
| <b>EL Pipeline Implementation (Part 1)</b>  |             |            |
| Working modular pipeline                    | 15%         | 15         |
| Data extraction from 3+ sources             | 10%         | 10         |
| Proper data storage (formats, organization) | 10%         | 10         |
| <b>Transformation and Analysis (Part 2)</b> |             |            |
| Data cleaning                               | 10%         | 10         |
| Appropriate visualizations and insights     | 10%         | 10         |
| Code quality and documentation              | 10%         | 10         |
| <b>Reflective Analysis (All Parts)</b>      |             |            |
| Quality of responses to questions           | 15%         | 15         |
| Pipeline Diagram                            | 10%         | 10         |
| <b>Overall Project Cohesion</b>             |             |            |
| Thematic consistency and narrative          | 5%          | 5          |
| Professional presentation and structure     | 5%          | 5          |
| <b>Total</b>                                | <b>100%</b> | <b>100</b> |

## Submission Requirements

Submit the following as a single ZIP file or GitHub repository:

- (a) **Code:** All Python scripts/Jupyter notebooks
- (b) **Data:** A sample of your data (due to size constraints, submit only processed/cleaned data samples)
- (c) **Documentation:**
  - README.md with setup instructions
  - Answers to all questions (PDF or Markdown)
  - Pipeline diagram
- (d) **Report:** Brief summary (1 page max) of your thematic focus, key findings, and challenges encountered

## Submission Notes:

- Due: **February 16, 2026, 11:59 PM EST**
- Submit on LMS
- Include your name and student ID in all files

## Required Readings

- Fundamentals of Modern Data Pipelines
- Challenges in Data Engineering Systems
- Data Version Control for ML Pipelines
- Fundamentals of Data Engineering (Chapters 3-5)

## Academic Integrity

- All code and written responses must be your own work
- You may use and adapt code from documentation/tutorials with proper attribution
- Collaboration on conceptual understanding is encouraged, but implementation must be individual

— End of Assignment —